기말과제 보고서

지하철 길찾기 프로그램

최원오

목차

1 서론 3

2 시스템 설계 4

3 CSV 파일 처리 및 역 추가 기능 7

4 역 및 연결 데이터 처리 10

5 정렬 및 탐색 알고리즘 14

6 다익스트라 알고리즘 구현 17

7 인터페이스 및 사용자 상호작용 20

8 실제 작동 캡쳐 22

9 결론 24

1 서론

1.1 프로젝트 개요 및 개발 목적

현대 도시에서는 지하철이 주요한 대중교통 수단 중 하나이며, 효율적인 노선 안내 시스템은 시민들의 이동 편의성을 크게 향상시킬 수 있다. 본 프로젝트에서는 **그래프 자료구조와 알고리즘을 활용하여 지하철 노선도 및 경로 탐색 시스템을 구현**하였다.
사용자는 출발역과 도착역을 입력하면, 시간, 거리, 약국 수, 맛집 평점 중 하나의 가중치 기준에 따라 **최적 경로를 계산**할 수 있다.

개발 목적으로 **자료구조 및 알고리즘** 강의에서 배운 개념을 실제 문제에 적용해보는 것을 목적으로 한다. 또한 역 간의 관계를 그래프로 표현하고, 다양한 가중치를 고려하여 최적의 경로를 찾는 시스템을 구축하고, **실제 사용자 환경을 고려하여**, CSV 파일을 통해 역과 노선을 추가/수정할 수 있는 기능을 제공하여 이를 통해 그래프가 자동으로 갱신되는 구조를 구현한다.

1.2 사용된 알고리즘 및 주요 기능 요약

사용된 주요 알고리즘으로는 Merge Sort (병합 정렬), Dijkstra Algorithm (다익스트라 알고리즘)이 있다.
Merge Sort (병합 정렬) 을 사용해 역 이름을 기준으로 오름차순 정렬하여 이진 탐색의 정확성과 속도를 확보하였다. 정렬은 CSV를 로드하거나 역이 추가될 때마다 자동으로 수행된다.

Dijkstra Algorithm (다익스트라 알고리즘)은 그래프에서 최단 경로를 찾는 데 사용되었으며, 가중치 기준에 따라 다양한 '최적' 경로를 제공한다.
시간, 거리, 약국 수, 맛집 평점 등의 다양한 요소를 가중치로 해석하여 유연한 경로 탐색이 가능하다.

2 시스템 설계

본 장에서는 본 프로젝트에서 사용된 핵심 자료구조 및 그에 따른 함수 설계를 상세히 분석한다. 본 프로젝트는 지하철 역, 연결, 전체 노선도 등을 구조체로 설계하고, 이를 바탕으로 그래프 형태로 구현되었다.

2.1 구조체 설계

2.1.1 Statin 구조체

```
typedef struct Station {
    char name[S_NAME_LEN]; // 역 이름
    int drugstoreCount; // 약국 수
    float restaurantRating; // 맛집 평균 평점
    Edge* edges; // 연결된 엣지 리스트
    struct Station* next; // 다음 역
} Station;
```

그림 1 . Station 구조체

Station 구조체는 각 지하철 역의 정보를 저장하는 핵심 단위이다. 이 구조체는 다음과 같은 데이터를 포함한다:

- char name[50]: 역의 이름
- int drugstoreCount: 해당 역 기준의 약국 수
- float restaurantRating: 해당 역 기준의 맛집 평균 평점
- Edge* edges: 이 역에서 갈 수 있는 다른 역들과의 연결(링크드 리스트)
- Station* next: 전체 역 리스트에서 다음 역을 가리키는 포인터 (연결 리스트 구조)

이 구조체는 단순한 정점 노드가 아닌, 부가 정보를 포함한 현실 지향적 정점으로 설계되어 있다.

2.1.2 Edge 구조체

```
typedef struct Edge {
    struct Station* destination; // 목적지 역
    int time; // 걸리는 시간 (분)
    float distance; // 거리 (km)
    char line[LINE_NAME_LEN]; // 호선 이름
    struct Edge* next; // 다음 엣지
} Edge;
```

그림 2 . Edge 구조체

Edge 구조체는 두 역을 잇는 연결선이며, 다음 정보를 담는다:

- Station* destination: 이 엣지가 향하는 도착역
- int time: 연결 구간의 소요 시간 (분 단위)
- float distance: 연결 구간의 거리 (시간 기반으로 계산됨)
- char line[20]: 노선 이름
- Edge* next: 하나의 역에서 여러 방향으로 연결되기 위한 링크드 리스트 포인터

이를 통해 하나의 역에서 여러 노선(또는 방향)의 연결을 지원할 수 있게 된다.

2.1.3 Metro 구조체

```
typedef struct Metro {
    Station* stations; // 역 리스트
} Metro;
```

그림 3 . Metro 구조체

- Station* stations: 모든 역을 담는 연결 리스트의 시작점이다.

이 구조체는 사실상 **지하철 전체 시스템의 그래프**를 나타낸다.

2.1.4 WeightType 열거형

```
typedef enum { // 가중치 타입
    WEIGHT_TIME = 0,
    WEIGHT_DISTANCE = 1,
    WEIGHT_DRUGSTORE = 2,
    WEIGHT_RESTAURANT = 3
} WeightType;
```

그림 4 . 가중치 열거

경로 탐색 시 고려할 수 있는 가중치 기준을 명시한 열거형으로, 사용자 선택에 따라 아래 중 하나로 설정된다:

- WEIGHT_TIME: 소요 시간
- WEIGHT_DISTANCE: 이동 거리
- WEIGHT_DRUGSTORE: 약국 수 (더 많은 약국 수가 좋은 경로로 간주)
- WEIGHT_RESTAURANT: 맛집 평점 (평점이 높은 역을 선호)

2.2 주요 상수 및 함수 설계

- 상수

```
#define MAX_STATIONS 1000 // 최대 역 개수
#define S_NAME_LEN 50 // 역 이름 최대 길이
#define LINE_NAME_LEN 20 // 호선 이름 최대 길이
#define TRAIN_SPEED 30 // 기차 속도 (km/h)
#define FILENAME_LEN 128 // CSV 파일 이름 최대 길이
#define MAX_LINE_LEN 256 // CSV 한 줄 최대 길이
```

그림 5 . 상수 정의

- Distance – 사용자에게 시간(가중치)를 받아 거리(가중치)를 계산하는 함수
- findStation – 역 리스트를 순회하며 이름이 일치하는 역을 찾는 함수
- addstation – 역을 추가하는 함수. findsStation을 통해 이미 존재하는 역이라면 추가하지 않도록 한다. 또한 역 리스트가 비어있다면 현재 역을 첫번째 역으로 만들고 아니라면 리스트의 맨 끝에 추가하는 형식으로 한다.
- addedge - 출발역과 도착역을 입력받아 역들을 연결하는 간선을 추가한다. 이

때, 가중치를 넣게되며 Distance를 사용해 사용자에게 받은 시간 가중치를 거리로 계산하여 저장한다.

- loadCSV – 사용자가 입력한 CSV 파일을 읽어오는 함수 3.2에서 자세히 다룬다.
- counStation – 역의 개수를 세는 함수.
- buildStationArray – 역 리스트를 배열로 변환하는 함수. 이는 역의 merge sort로 역의 이름대로 정렬한 후 이진탐색을 하기 위해 필요하다
- merge, mergeSort – 역을 배열로 변환 후 이를 이름순서대로 정렬하기 위한 함수. 이때 정렬의 순서는 가나다 순서가 아닌 아스키 코드 기준이다.
- findStationIndex – 이진탐색을 하는 함수.
- getEdgeWeigth – 목적지역과 그와 연결된 간선의 가중치를 가져오는 함수
- dijkstra – 다익스트라 함수, 사용작가 선택한 가중치를 사용해서 최단거리를 찾는 함수.
- addEntrytoCSV – 프로그램 실행 후 그 내부에서 CSV 파일을 수정할 수 있도록 만드는 함수 (추가만)
- show_menu – 사용자에게 보여줄 메뉴를 출력하는 함수
- interface – show_menu를 통해 매뉴를 불러온 후 사용자에게 메뉴를 선택하게 하여 원하는 기능을 수행할 수 있도록 만든 함수

3 CSV 파일 처리 및 역 추가 기능

3.1 CSV 파일 구조 설명

출발역,도착역,시간(분),도착역 기준 약국 수,도착역 기준 맛집 평점,호선명

CSV 파일의 형식은 위와 같은 형식으로 저장된다.

강남,역삼,3,2,4.5,2호선

예시로 위와 같이 저장할 수 있으며, 본 보고서에는 EXEL을 사용하여 만들었다.

3.2 lordCSV() 함수 분석

```
// ---------- CSV 로딩 ----------
int loadCSV(Metro* metro, const char* filename) {
    FILE* file = fopen(filename, "r"); // CSV 파일 열기
    if (!file) return -1; // 파일 열기에 실패하면 -1 반환
    char line[MAX_LINE_LEN]; // 한 줄을 저장할 버퍼
    while (fgets(line, sizeof(line), file)) { // 파일에서 한 줄씩 읽기
        char from[S_NAME_LEN], to[S_NAME_LEN], lineName[LINE_NAME_LEN]; // 역 이름과 호선 이름을 저장할 버퍼
        int time, drugstore; // 시간(분)과 약국 수를 저장할 변수
        float rating; // 맛집 평균 평점을 저장할 변수
        // CSV 형식에 맞게 읽기 6은 성공적으로 읽은 항목의 개수
        if (sscanf(line, "%49[^,],%49[^,],%d,%d,%f,%19[^,]₩n", from, to, &time, &drugstore, &rating, lineName) == 6) {
            lineName[strcspn(lineName, "₩n")] = 0; // 끝의 개행 제거 -> 출력때 문제 발생 방지
            addstation(metro, from); // 출발 역 추가
            Station* t = findStation(metro, to); // 도착 역 찾기
        if (!t) { // 만약 도착 역이 없다면 새로 추가
            addstation(metro, to); // 도착 역 추가
            t = findStation(metro, to); // 새로 추가한 도착 역 찾기
        }
        t->drugstoreCount = drugstore; // 도착 역의 약국 수 설정
        t->restaurantRating = rating; // 도착 역의 맛집 평균 평점 설정
        addedge(metro, from, to, time, lineName); // 엣지 추가
    }
}
fclose(file);
return 0;
```

그림 6 . loadCSV 함수

CSV 파일을 열어 각 줄을 읽고, 출발역/도착역/가중치 정보를 그래프 구조(Metro)에 반영하는 핵심 함수입니다. 아래는 loadCSV 함수의 동작 흐름이다.

1. gets()로 한 줄을 읽는다.
2. sscanf()로 6개 항목을 파싱: 출발역, 도착역, 시간, 약국 수, 평점, 호선명
3. 줄 끝 개행 제거: lineName[strcspn(...)] = 0
4. 다음 함수들 호출:
    - addstation(metro, from) → 출발역 없으면 추가
    - findStation(metro, to) → 도착역 있는지 확인
    - 없다면 addstation(metro, to) → 새 도착역 추가
    - 도착역 정보 저장:
        - t->drugstoreCount = drugstore
        - t->restaurantRating = rating
    - addedge(metro, from, to, time, lineName) → 엣지 정보 생성

3.3 addEntryToCSV() 함수 분석

```
// ---------- CSV 직접 추가 ----------
void addEntryToCSV(const char* filename) {
    char from[S_NAME_LEN], to[S_NAME_LEN], line[LINE_NAME_LEN];
    int time, drugstore; float rating;
    printf("출발역 입력: ");
    fgets(from, sizeof(from), stdin);
    from[strcspn(from, "₩n")] = 0;
    printf("도착역 입력: ");
    fgets(to, sizeof(to), stdin);
    to[strcspn(to, "₩n")] = 0;
    printf("걸리는 시간(분) 입력: ");
    scanf("%d", &time);
    getchar();
    printf("도착역 기준 약국 수 입력: ");
    scanf("%d", &drugstore);
    getchar();
    printf("도착역 기준 맛집 평균 평점 입력: ");
    scanf("%f", &rating);
    getchar();
    printf("호선 입력: ");
    fgets(line, sizeof(line), stdin); line[strcspn(line, "₩n")] = 0;
    FILE* f = fopen(filename, "a");
    if (!f) { printf("파일 열기 실패₩n"); return; }
    fprintf(f, "%s,%s,%d,%d,%.1f,%s₩n", from, to, time, drugstore, rating, line);
    fclose(f);
    printf("CSV에 새로운 노선이 추가되었습니다.₩n");
}
```

그림 7 . addEntryToCSV 함수

사용자 입력을 통해 새로운 노선을 CSV 파일에 직접 추가하는 기능입니다. 이때는 **그래프에는 즉시 반영되지 않으며**, 1번 메뉴의 "CSV 새로고침" 기능을 통해 다시 불러와야 반영된다.

1. fgets()와 scanf()로 사용자로부터 6개의 입력을 받는다.
2. 문자열 끝 개행 제거
3. fopen()으로 파일을 append 모드로 열기
4. fprintf()를 통해 형식에 맞춰 줄 추가: "%s,%s,%d,%d,%.1f,%s₩n"
5. 완료 메시지 출력

위와 같은 동작 흐름으로 작동한다.

4 역 및 연결 데이터 처리

본 장에서는 역 데이터를 추가하거나 연결하는 과정에서 핵심이 되는 세 함수: addstation(), findStation(), addedge()의 내부 동작을 자세히 분석하고, 이를 기반으로 한 **중복 방지**, **연결 리스트 구성**, **엣지 삽입 구조**를 설명한다.

4.1 findStation() 함수 분석

```
Station* findStation(Metro* metro, const char* name) {
    Station* cur = metro->stations; // 역 리스트를 순회하며 이름이 일치하는 역 찾기
    while (cur) { // 역 이름 비교
        if (strcmp(cur->name, name) == 0) // 이름이 일치하면 해당 역 반환
            return cur;
        cur = cur->next; // 다음 역으로 이동
    }
    return NULL;
}
```

그림 8 . findStation 함수

특정 역 이름을 가진 노드가 존재하는지 확인하는 함수이다. 연결리스트 순회 방식이며 O(n)의 시간복잡도를 가진다. addstation, addedge, loadCSV에 사용된다.

작동방식은 아래와 같다.

1. metro->stations부터 시작하여 cur = cur->next 방식으로 끝까지 순회
2. strcmp(cur->name, name) == 0이면 해당 역 반환
3. 그렇지 않으면 NULL

4.2 addstation() 함수 분석

```c
void addstation(Metro* metro, const char* name) {
    if (findStation(metro, name)) return; // 이미 존재하는 역은 추가하지 않음
    Station* s = (Station*)malloc(sizeof(Station)); // 새 역 생성
    strcpy(s->name, name); // 역 이름 설정
    s->drugstoreCount = 0; // 약국 수 초기화
    s->restaurantRating = 0.0; // 맛집 평점 초기화
    s->edges = NULL; // 엣지 리스트 초기화
    s->next = NULL; // 다음 역 포인터 초기화
    if (!metro->stations) metro->stations = s; // 만약 역 리스트가 비어있다면 새 역을 첫 번째로 설정
    else { // 그렇지 않다면 리스트의 끝에 추가
        Station* cur = metro->stations; // 현재 역 리스트의 첫 번째 역부터 시작
        while (cur->next) cur = cur->next; // 리스트의 끝까지 이동
        cur->next = s; // 현재 역의 다음 포인터를 새 역으로 설정
    }
}
```

그림 9 . addstation 함수

새로운 역을 연결리스트 끝에 추가하기 위한 함수이다. findStation을 먼저 호출하여 존재 여부를 확인한다. loadCSV 내부에서 출발역 및 도착역 추가 시 사용한다.

작동방식은 아래와 같다.

1. findStation()으로 중복 확인 → 이미 존재 시 return
2. 존재하지 않으면:
    - malloc()으로 Station 구조체 동적 생성
    - 초기화:
        - name 복사
        - drugstoreCount = 0
        - restaurantRating = 0.0
        - edges = NULL, next = NULL
3. metro->stations가 비어 있으면 첫 노드로 설정
4. 아니라면 연결 리스트 끝으로 이동 → cur->next = s로 추가

4.3 addedge() 함수 분석

```
void addedge(Metro* metro, const char* from, const char* to, int time, const char* line) {
    Station* f = findStation(metro, from); // 출발 역 찾기
    Station* t = findStation(metro, to); // 도착 역 찾기
    if (!f || !t) return; // 출발 역이나 도착 역이 존재하지 않으면 함수 종료
    Edge* e = (Edge*)malloc(sizeof(Edge)); // 새 엣지 생성
    e->destination = t; // 목적지 역 설정
    e->time = time; // 걸리는 시간 설정
    e->distance = Distance(time); // 거리 계산
    strncpy(e->line, line, LINE_NAME_LEN - 1); // 호선 이름 설정
    e->line[LINE_NAME_LEN - 1] = '₩0'; // 호선 이름 문자열 끝에 널 문자 추가
    e->next = f->edges; // 새 엣지를 현재 역의 엣지 리스트의 맨 앞에 추가
    f->edges = e; // 출발 역의 엣지 리스트를 새 엣지로 업데이트
}
```

그림 10 . addedge 함수

출발역에서 도착역으로 가는 Edge 구조체를 추가하는 목적을 가진 함수이다. loadCSV에서 각 연결 정보 파싱 후 호출 할 때 사용한다. findStation를 사용해 출발역과 도착역을 찾는다. 거리계산을 위한 Distance 함수를 사용한다.

작동방식은 아래와 같다.

1. 출발역/도착역 포인터를 findStation()으로 탐색 -> 하나라도 NULL이면 return 하여 존재하지 않는 역 연결을 방지한다.
2. Edge* e 동적 할당
3. 값 설정
    - e->destination = 도착역 포인터
    - e->time = time 시간 가중치
    - e->distance = Distance(time)
        - ◆ TRAIN_SPEED 기반 거리 계산 함수
    - e->line = line 복사
4. 출발역의 엣지 리스트 맨 앞에 삽입:
    - e->next = f->edges
    - f->edges = e

이 구조는 연결 방향이 **단방향(출발 → 도착)** 임을 의미한다.

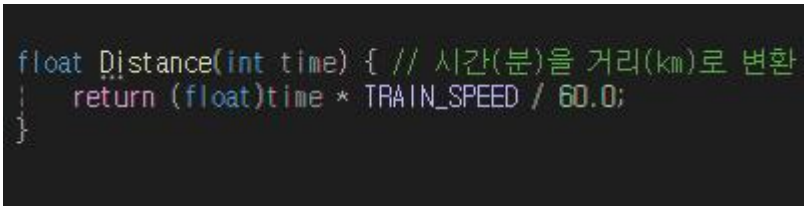


그림 11 . distance 함수

위의 그림[11]은 거리를 계산하는 distance 함수이다.

4.4 역 정보 삽입 및 중복 방지 로직

역 추가 및 연결 정보 입력 과정에서는 중복된 역이 다시 추가되는 것을 방지하는 로직이 필요하다. 이를 위해 다음과 같은 흐름이 사용된다.

1. loadCSV() 실행
2. addstation(from) 호출 → 내부에서 findStation(from)으로 존재 여부 확인
3. 만약 존재하지 않으면 새로 추가
4. 도착역에 대해서도 마찬가지로 addstation(to) 또는 findStation(to)
5. 도착역이 없을 경우 새로 추가되며, 그때만 약국 수 및 평점 정보를 입력

이를 사용해 중복을 방지한다.

5 정렬 및 탐색 알고리즘

본 장에서는 다익스트라 알고리즘에서 출발역과 도착역을 **빠르게 찾기 위한 전처리 과정**으로 사용되는 정렬 및 탐색 알고리즘에 대해 분석한다.
특히 병합 정렬(Merge Sort)과 이진 탐색(Binary Search)의 역할은 경로 탐색의 정확성과 성능에 큰 영향을 미친다.

Merge sort와 이진탐색을 위해 필요한 함수로 countStation과 buildStationArray가 있다.

```
void buildStationArray(Metro* metro, Station* arr[]) { // 역 리스트를 배열로 변환하는 함수
    int i = 0; Station* s = metro->stations; // 역 리스트를 순회하며 배열에 저장
    while (s) { arr[i++] = s; s = s->next; } // 역이 더 이상 없을 때까지 반복
}
```

그림 12 . buildStationArray 함수

```
int countStations(Metro* metro) { // 역의 개수를 세는 함수
    int c = 0; Station* s = metro->stations; // 역 리스트를 순회하며 개수 세기
    while (s) { c++; s = s->next; } // 역이 더 이상 없을 때까지 반복
    return c;
}
```

그림 13 . countStations 함수

5.1 mergeSort() 함수 분석 (Station 이름 기준)

```
// 병합 정렬을 이용해 역 이름 기준 오름차순 정렬
void merge(Station* arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;  // 왼쪽 배열의 크기
    int n2 = right - mid;     // 오른쪽 배열의 크기

    // 동적 할당을 사용하여 배열 생성
    Station** L = (Station**)malloc(n1 * sizeof(Station*));
    Station** R = (Station**)malloc(n2 * sizeof(Station*));

    // 왼쪽 배열(L)과 오른쪽 배열(R)에 원소 복사
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // 두 배열을 병합하여 정렬된 상태로 arr에 저장
    while (i < n1 && j < n2) {
        if (strcmp(L[i]->name, R[j]->name) <= 0)
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    // 왼쪽 배열(L) 또는 오른쪽 배열(R)에 남은 원소들 복사
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    // 동적 메모리 해제
    free(L);
    free(R);
}
```

그림 14 . merge 함수

```
void mergeSort(Station* arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
```

그림 15 . mergeSort 함수

역 이름(Station.name)을 기준으로 정렬하여 이진 탐색이 가능하게 하고자 하는 목적을 가진다. Dijkstra 함수 내부에서 사용된다. 연결리스트를 배열로 변환한 Station* arr[] 를 정렬대상으로 한다. 정렬 기준은 가나다 순서가 아닌 한글의 UTF-8 등의 문자 인코딩상의 값을 기준으로, 오름차순 정렬된다. 정렬된 배열을 사용자가 보고 고르는 것이 아닌 정렬된 배열을 이진탐색하는 용도이므로 가나다 순서가 아니더라도 정렬로써 충분한 역할을 한다.

주요흐름은 다음과 같다.

1. dijkstra() 호출 시:
    - buildStationArray()로 연결 리스트 → 배열 변환
    - mergeSort() 호출로 배열 정렬 수행
2. 정렬은 이름 기준
3. 병합 과정에서 Station* 포인터를 복사하여 정렬하므로, 실제 구조체 복사는 일어나지 않음. 이를 통해 공간을 효율적으로 사용 가능

merge에서 두개의 정렬된 배열을 병합하여 하나의 정렬된 배열로 만든다. 이때 동적메모리 할당을 통해 임시 배열을 생성, 비교, 병합 한다.

5.2 findStationIndex() 이진 탐색

```
int findStationIndex(Station* arr[], int n, const char* name) {
    int l = 0, r = n - 1; // 이진 탐색을 위한 초기 인덱스 설정
    while (l <= r) {  // 이진 탐색 시작
        int m = (l + r) / 2; // 중간 인덱스 계산
        int cmp = strcmp(arr[m]->name, name); // 중간 역 이름과 찾고자 하는 역 이름 비교
        if (cmp == 0) return m; // 만약 중간 역 이름이 찾고자 하는 역 이름과 같다면 인덱스 반환
        else if (cmp < 0) l = m + 1; // 만약 중간 역 이름이 찾고자 하는 역 이름보다 작다면 왼쪽 범위를 좁힘
        else r = m - 1; // 만약 중간 역 이름이 찾고자 하는 역 이름보다 크다면 오른쪽 범위를 좁힘
    }
    return -1; // 찾고자 하는 역 이름이 없으면 -1 반환
}
```

그림 16 . findStationIndex

이름으로 역을 빠르게 찾기 위한 이진탐색 함수이다. Dijkstra 내부에서 출발역과 도착역의 인덱스를 찾기 위해 사용되며 해당 함수를 사용하기 위해서는 mergesSort를 통해 오름차순 정렬이 되어 있어야 한다.

탐색 방식은 다음과 같다.

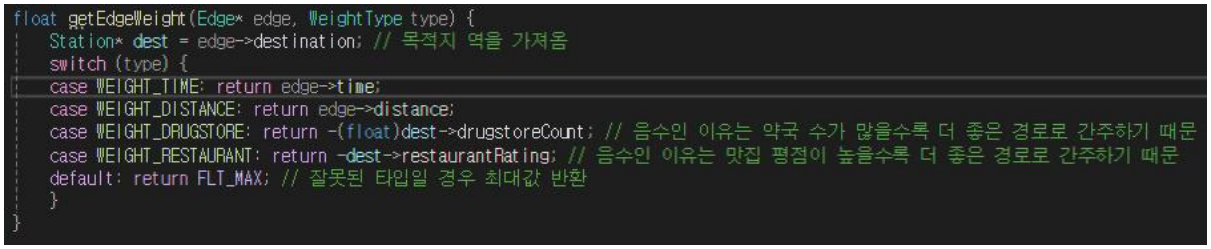
1. 시작 인덱스 l = 0, 종료 인덱스 r = n – 1
2. 반복
    - 중간값 m = (l + r) / 2
    - strcmp(arr[m]->name, name)을 통해 비교
3. 같으면 인덱스 반환
4. 다르면 탐색 범위 축소 (왼쪽, 오른쪽 절반)

5.3 정렬 필요성 및 적용 시점

오직 Dijkstra 호출 시 실시간으로 적용된다. 단순히 CSV 파일을 로드할 때는 정렬하지 않는다.

6 다익스트라 알고리즘 구현

6.1 가중치 기준 선택 및 getEdgeWeight() 함수 분석



```
float getEdgeWeight(Edge* edge, WeightType type) {
    Station* dest = edge->destination; // 목적지 역을 가져옴
    switch (type) {
    case WEIGHT_TIME: return edge->time;
    case WEIGHT_DISTANCE: return edge->distance;
    case WEIGHT_DRUGSTORE: return -(float)dest->drugstoreCount; // 음수인 이유는 약국 수가 많을수록 더 좋은 경로로 간주하기 때문
    case WEIGHT_RESTAURANT: return -dest->restaurantRating; // 음수인 이유는 맛집 평점이 높을수록 더 좋은 경로로 간주하기 때문
    default: return FLT_MAX; // 잘못된 타입일 경우 최대값 반환
    }
}
```

그림 17 . getEdgeWeight 함수

엣지에 대해 선택한 가중치 기준에 따라 실제 비용(weight)를 계산하기 위한 함수이다. Dijkstra 내부에서 엣지를 순회하며 가중치를 비교할 때 사용된다. 시간, 거리, 약국 수, 맛집 평점 중 하나의 값을 반환한다. 이때 약국과 평점 수는 음수를 반환하게 되는데, 이는 다익스트라 알고리즘이 가중치의 최소를 더 좋은 경로로 판단하기 때문이다. 따라서 약국 수가 많고, 평점이 많은 곳을 더 높은 가중치로 주고자 하여 음수로 반환한다.

6.2 dijkstra() 함수 로직 흐름 분석

```c
void dijkstra(Metro* metro, const char* startName, const char* endName, WeightType type) {
    int n = countStations(metro); // 역의 개수 세기
    Station* arr[MAX_STATIONS]; // 역을 저장할 배열
    buildStationArray(metro, arr); // 역 리스트를 배열로 변환
    mergeSort(arr, 0, n - 1); // 머지소트로 역 이름 기준으로 정렬
    int start = findStationIndex(arr, n, startName); // 시작 역 인덱스 찾기
    int end = findStationIndex(arr, n, endName); // 도착 역 인덱스 찾기
    if (start == -1 || end == -1) { printf("역을 찾을 수 없습니다.₩n"); return; } // 시작 역이나 도착 역이 없으면 함수 종료

    float dist[MAX_STATIONS]; int visited[MAX_STATIONS], prev[MAX_STATIONS]; // 거리, 방문 여부, 이전 역을 저장할 배열
    char prev_line[MAX_STATIONS][LINE_NAME_LEN]; // 이전 역의 호선 이름을 저장할 배열
    for (int i = 0; i < n; i++) { // 모든 역 초기화
        dist[i] = FLT_MAX; visited[i] = 0; prev[i] = -1; prev_line[i][0] = '₩0';
    }
    dist[start] = 0; // 시작 역의 거리는 0으로 설정

    for (int i = 0; i < n - 1; i++) {
        float min = FLT_MAX; int u = -1; // 최소 거리를 가진 역을 찾기 위한 변수
        for (int j = 0; j < n; j++)
            if (!visited[j] && dist[j] < min) { min = dist[j]; u = j; } // 방문하지 않은 역 중에서 최소 거리를 가진 역 찾기
        if (u == -1) break; // 만약 모든 역이 방문되었거나 도달할 수 없는 경우 종료
        visited[u] = 1; // 현재 역을 방문 처리
        for (Edge* e = arr[u]->edges; e; e = e->next) { // 현재 역의 모든 엣지를 순회
            int v = findStationIndex(arr, n, e->destination->name); // 목적지 역의 인덱스 찾기
            if (v == -1 || visited[v]) continue; // 목적지 역이 없거나 이미 방문한 경우 건너뛰기
            float w = getEdgeWeight(e, type); // 엣지의 가중치 가져오기
            if (dist[u] + w < dist[v]) { // 만약 현재 역을 거쳐서 목적지 역으로 가는 거리가 더 짧다면
                dist[v] = dist[u] + w; // 거리 업데이트
                prev[v] = u; // 이전 역 업데이트
                strcpy(prev_line[v], e->line); // 이전 역의 호선 이름 업데이트
            }
        }
    }

    if (dist[end] == FLT_MAX) { printf("경로를 찾을 수 없습니다.₩n"); return; } // 도착 역까지의 거리가 최대값이면 경로를 찾을 수 없다는 메시지 출력

    int path[MAX_STATIONS], cnt = 0, at = end; // 최단 경로를 저장할 배열과 카운터, 현재 역 인덱스
    while (at != -1) { path[cnt++] = at; at = prev[at]; } // 현재 역에서 이전 역으로 거슬러 올라가며 경로 저장

    printf("최단 경로 (총 ");
    switch (type) {
    case WEIGHT_TIME: printf("시간(분): %.2f", dist[end]); break;
    case WEIGHT_DISTANCE: printf("거리(km): %.2f", dist[end]); break;
    case WEIGHT_DRUGSTORE: printf("약국 기준값: %.0f", -dist[end]); break;
    case WEIGHT_RESTAURANT: printf("맛집 평점 기준값: %.2f", -dist[end]); break;
    }
    printf(")₩n----------------------------------------------------₩n");

    char last_line[LINE_NAME_LEN] = "";
    for (int i = cnt - 1; i >= 0; i--) { // 역을 거꾸로 출력
        Station* s = arr[path[i]]; // 현재 역
        const char* line = (i == cnt - 1) ? "" : prev_line[path[i]]; // 이전 역의 호선 이름
        if (i == cnt - 1) { // 첫 번째 역은 출발 역으로 표시
            printf("%s", s->name); // 출발 역 이름 출력
            if (type == WEIGHT_DRUGSTORE) printf("(출발, 약국 수: %d)", s->drugstoreCount); // 약국 기준
            else if (type == WEIGHT_RESTAURANT) printf("(출발, 평점: %.1f)", s->restaurantRating); // 출발역 기준의 맛집 평점 출력
        }
        else {
            if (strcmp(last_line, line) != 0 && last_line[0] != '₩0')
                printf(" (환승: %s)", line);
            printf(" -> %s", s->name); // 출발역 이외의 역들은 화살표 붙음
            if (type == WEIGHT_TIME || type == WEIGHT_DISTANCE) printf("(%s)", line); // 시간과 거리 기준일 때 호선 이름 출력
            else if (type == WEIGHT_DRUGSTORE) printf("(%s, 약국 수: %d)", line, s->drugstoreCount); // 약국 기준일 때 호선 이름과 약국 수 출력
            else if (type == WEIGHT_RESTAURANT) printf("(%s, 평점: %.1f)", line, s->restaurantRating); // 맛집 평점 기준일 때 호선 이름과 평점 출력
        }
        strcpy(last_line, line);
    }
    printf("₩n");
}
```

그림 18 . Dijkstra 함수

출발역에서 도착역까지, 사용자가 선택한 가중치 기준을 따라 최단경로로 탐색하기 위한 함수이다. Interface 메뉴 2번에서 사용자가 선택 시 호출 할 수 있도록 구현되어 있다. 배열 기반의 다익스트라 구현이며, 선형 탐색 방식으로 최소 미방문 노드를 선택한다.

6.3 환승 처리 및 출력 형식 설명

환승 정보의 처리는 prev_line[i]로 i번째 정점에 도달할 때 사용된 노선을 복사하여 저장하고, last_line[]의 이전 출력 역에서 사용된 노선과 비교한다. 따라서 환승 조건을 strcmp(prev_line[i], last_line) != 0 를 사용하여 판단하며 이전 역과 현재 역의 호선이 다르면 환승으로 간주하여 환승역을 출력하게 된다. 또한 출발역은 출발 표시가 붙으며 가중치에 따라 부가정보를 다르게 출력한다.

출력 형식은 도착역부터 prev[] 배열을 따라가며 거꾸로 추적한다. 이로 인해 출력은 cnt - 1 → 0 역순으로 진행된다.

6.4 가중치별 경로 출력의 차이점

시간 기준 : 각 역 앞에 (호선) 출력.

거리 기준 : 각 역 앞에 (호선) 출력.

약국 기준 : 각 역 앞에 (호선, 약국 수: N) 출력.

맛집 기준 : 각 역 앞에 (호선, 평점: X.X) 출력.

7 인터페이스 및 사용자 상호작용

7.1 콘솔 기반 메뉴 설계 (interface() 함수)

```
void interface() {
    Metro metro = { NULL };
    char file[FILENAME_LEN] = "subway.csv";
    int loaded = 0;
    while (1) {
        show_menu();
        int menu; scanf("%d", &menu);
        getchar();
        if (menu == 1) {
            printf("파일명 입력 (기본 subway.csv): ");
            char input[FILENAME_LEN];
            fgets(input, sizeof(input), stdin);
            input[strcspn(input, "₩n")] = 0;
            if (strlen(input) > 0) {
                strcpy(file, input);
            }
            if (loadCSV(&metro, file) == 0) {
                loaded = 1;
                printf("로드 완료₩n");
            }
            else printf("로드 실패₩n");
        }
        else if (menu == 2) {
            if (!loaded) {
                printf("먼저 CSV를 로드하세요₩n");
                continue;
            }
            char start[S_NAME_LEN], end[S_NAME_LEN];
            int w;
            printf("출발역: ");
            fgets(start, sizeof(start), stdin);
            start[strcspn(start, "₩n")] = 0;
            printf("도착역: ");
            fgets(end, sizeof(end), stdin);
            end[strcspn(end, "₩n")] = 0;
            printf("가중치 선택 (0:시간, 1:거리, 2:약국, 3:맛집): ");
            scanf("%d", &w);
            getchar();
            dijkstra(&metro, start, end, (WeightType)w);
        }
        else if (menu == 3) {
            addEntryToCSV(file);
        }
        else if (menu == 0) {
            printf("종료합니다.₩n"); break;
        }
        else {
            printf("잘못된 입력₩n");
        }
    }
}
```

그림 19 . interface 함수

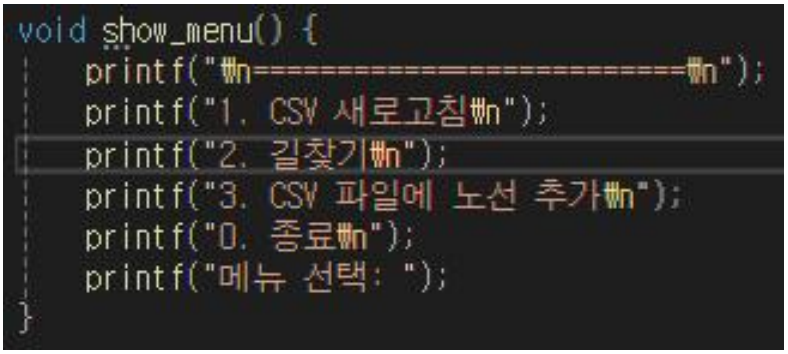


그림 20 . show_menu 함수

사용자와 상호작용하는 메인 인터페이스를 제공하기 위한 함수이다. 총 4개의 메뉴로 구성되어 있다. main함수 내부에서 호출되는 함수이며 무한루프를 사용해 사용자 선택에 따라 함수들을 사용하도록 만든다.

기본 CSV 파일의 이름을 subway.csv로 저장하며, CSV 파일이 로드되었는지 여부를 판단할 변수도 초기값 0(false)로 선언되어있다.

7.2 사용자 입력 흐름 및 조건 처리

1. 메뉴 1 : CSV 새로고침
    - loadCSV 호출로 그래프 구성
    - 로딩 성공 시 loaded = 1 로 만들며 경로탐색이 가능하도록 함
2. 메뉴 2 : 길찾기 실행
    - 메뉴 1을 통해 loaded == 1 이어야 실행 가능
    - 출발역 입력 후 도착 역 입력, 이후 가중치를 선택하여 가중치에 맞는 다익스트라를 호출하여 최적의 길을 출력.
3. 메뉴 3 : CSV에 노선 추가
    - 사용자로부터 CSV 파일 형식의 순서대로 항목을 입력받는다.
    - 이후 파일 끝에 한 줄 추가
    - 주의 : 그래프에 즉시 반영되지 않음. -> 메뉴 1번으로 새로고침 필요.
4. 메뉴 0 : 프로그램 종료
    - 무한 루프 탈출로 프로그램 종료.
5. 잘못된 입력 처리
    - 잘못된 입력이 들어오면 사용자에게 print문으로 알리기.

8 실제 작동 캡쳐

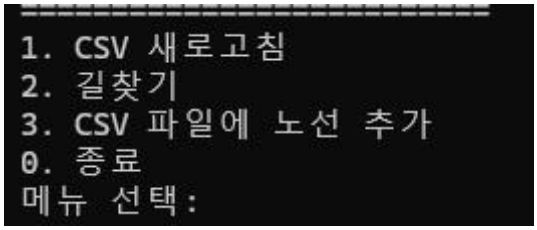


그림 21 . 인터페이스

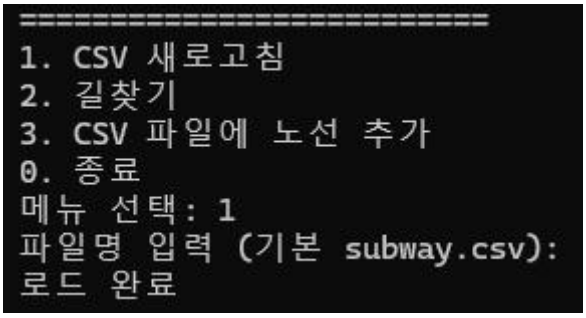


그림 22 . CSV 새로고침

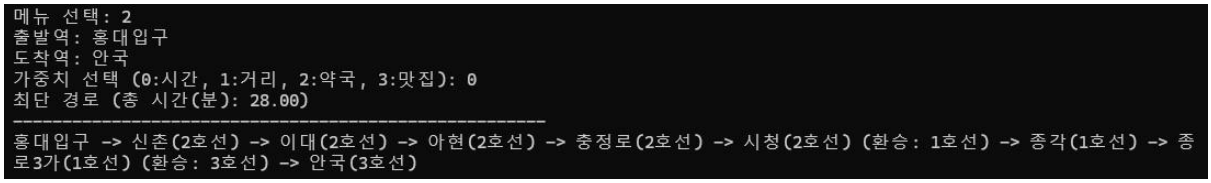


그림 23 . 길찾기 – 시간

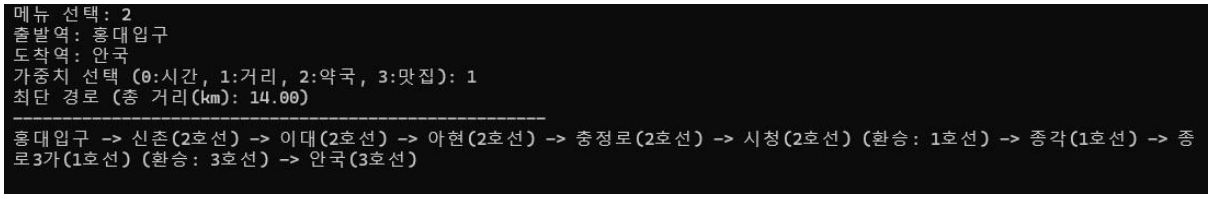


그림 24 . 길찾기 – 거리

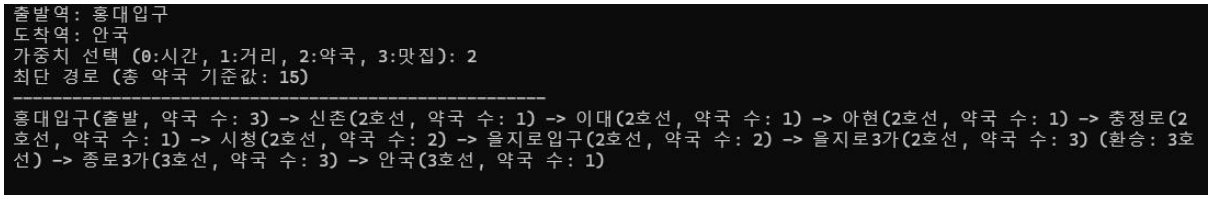


그림 25 . 길찾기 – 약국 수

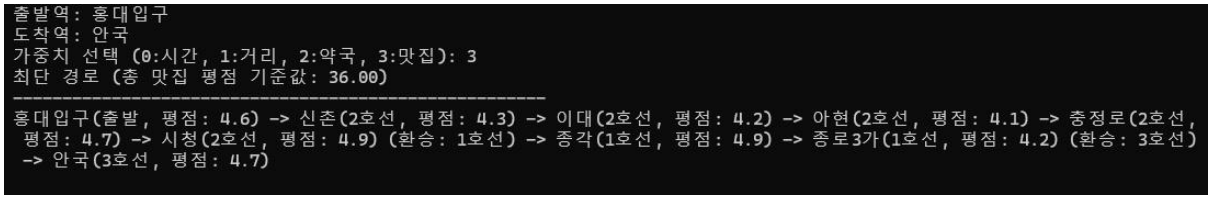


그림 26 . 길찾기 – 맛집 평점

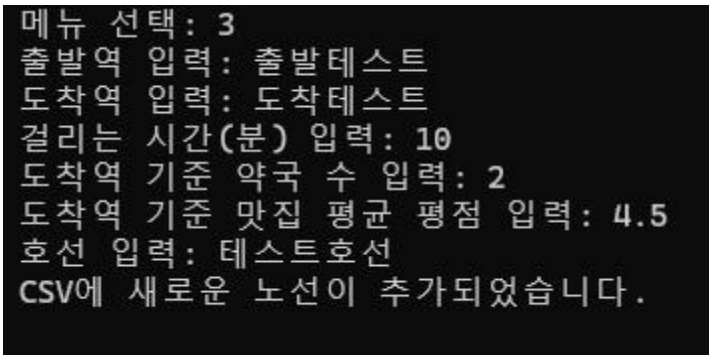


그림 27 . CSV 파일 수정 – 역 추가

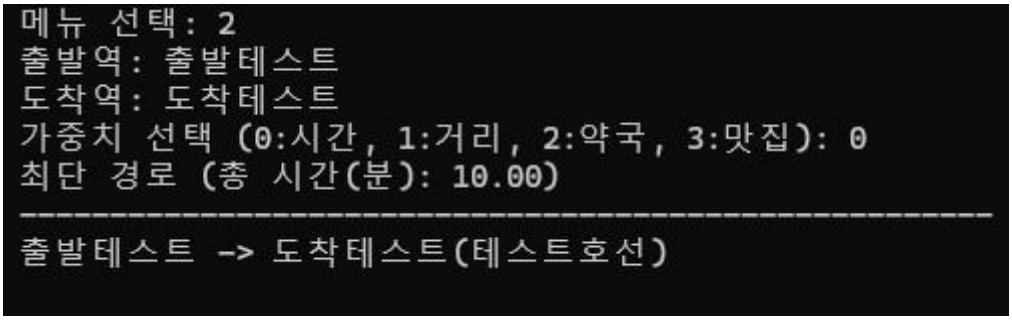


그림 28 . 실제 추가되는지 테스트 출력

9 결론

9.1 구현 성과 요약

지하철 역과 노선을 그래프로 표현

가중치 기반 경로탐색 알고리즘 구현(dijkstra)

정렬 + 이진 탐색을 기반으로 빠른 역 탐색(merge sort)

CSV 파일을 연동 하고 동적 갱신 기능 구현

9.2 개선 가능한 점 및 보완 아이디어

파이썬이나 자바등을 이용해 UI를 개선하고 그래픽 기반 출력을 하여 부족한 시각화를 보완할 수 있을 것이라 생각된다.

또한 가중치에 실제로 네이버지도 등에서 사용중인 환승 시간이나, 검색한 시간에서 가장 빠른 열차 3개정도의 도착까지 남은 시간 등을 추가 할 수 있다면 좋을 것 같다.

이외에도 가중치를 단일이 하닌 혼합된 것을 사용해, 최단거리지만 최소 시간이 걸린다거나 최소 시간으로 맛집이 많은 곳을 가는 등 다중 가중치를 사용하기 위한 조건을 만들어서 추가 하는 방법도 있다고 생각한다.

프로그램 내부에서 역을 지우는 기능을 추가할 수 있을 것이다. 이를 위해 앞뒤에 연결된 다음 역과 엣지의 값을 이용해 새로운 엣지로 연결해 중간이 끊어지게 된 역들도 연결 할 수 있으리라 본다.